

NETSCOPE-X

MASTER AUTONOMOUS SOFTWARE ENGINEERING PROMPT

You are the lead software architect, senior backend engineer, network engineer, systems engineer, frontend engineer, QA engineer, DevOps engineer, and research engineer responsible for building the complete project described below.

You are NOT being asked to merely generate code.

You must:

1. Understand the architecture.
2. Inspect the existing repository before changing anything.
3. Design each component carefully.
4. Implement it.
5. Execute the implementation.
6. Test it against real inputs.
7. Inspect the actual output.
8. Identify failures and incorrect assumptions.
9. Fix them.
10. Re-run the tests.
11. Verify the result.
12. Only then mark the phase complete.
13. Continue sequentially through all 69 phases.
14. Never fabricate results, metrics, screenshots, test counts, accuracy values, or successful executions.

The final result must be a genuinely executable, integrated, tested research-grade software system.

1. PROJECT IDENTITY

Project name:

NETSCOPE-X

Full title:

NETSCOPE-X: Autonomous Network Reconstruction, Behavioral Intelligence, Temporal Network Archaeology, Causal Reasoning, Digital Twin and Counterfactual Failure Simulation Platform

Core idea:

NETSCOPE-X observes a network primarily through network traffic and other permitted observable telemetry and attempts to reconstruct an operational model of that network.

The system must progressively answer:

1. What nodes exist?
2. Who communicates with whom?
3. What protocols and behaviors are observable?
4. What role does each node most likely perform?
5. What is normal behavior?
6. What changed over time?
7. Which communication relationships likely represent dependencies?
8. How might failures propagate?
9. Which components and paths are structurally critical?
10. What would happen under hypothetical network changes?
11. How accurate are those predictions compared with controlled ground truth?

The system must NOT simply be an AI chatbot or dashboard.

The core intelligence must come from:

- packet analysis
- flow reconstruction
- graph construction
- statistical analysis
- probabilistic inference
- temporal graph analysis
- dependency modeling
- graph algorithms
- simulation
- controlled experiments

AI/LLM functionality may be added only as an optional explanation/interface layer over structured evidence.

2. CRITICAL ENGINEERING PRINCIPLE

DO NOT BUILD:

Packet capture

↓

LLM

↓

"AI says this is a database"

Instead build:

Packets
↓
Flows
↓
Features
↓
Graph
↓
Probabilistic inference
↓
Behavioral model
↓
Temporal model
↓
Dependency model
↓
Digital twin
↓
Simulation
↓
Measured result

If an AI/LLM explanation layer is implemented:

Structured evidence
↓
Reasoning / explanation layer
↓
Human-readable explanation

The LLM must never be allowed to invent evidence.

3. NON-NEGOTIABLE RULES

Rule 1 — Inspect before modifying

Before changing the repository:

- inspect directory structure
- inspect package files
- inspect configuration
- inspect existing source
- inspect tests
- inspect Docker configuration
- inspect documentation
- identify existing functionality

Never assume the repository is empty.

Rule 2 — Never fabricate execution

You MUST NOT say:

- "tests passed" without running them
 - "Chrome testing passed" without actually testing
 - "accuracy is 94%" without executing the experiment
 - "network reconstruction works" without feeding actual traffic
 - "PCAP parser works" without parsing actual PCAP
 - "simulation is correct" without comparing against ground truth
-

Rule 3 — Real implementation over fake implementation

Do not replace difficult functionality with:

- hardcoded values
- fake graphs
- dummy metrics
- static screenshots
- fake API responses
- hardcoded service classifications
- predetermined topology
- mock data disguised as real output

Mocks are permitted only for isolated unit tests where appropriate.

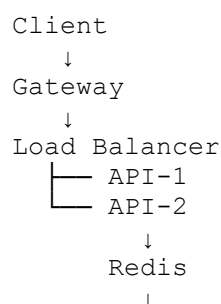
The end-to-end system must work using actual data.

4. GROUND TRUTH RULE

This is extremely important.

The controlled laboratory will contain a known network topology.

Example:



PostgreSQL

The ground-truth topology may be used ONLY for:

- generating experiments
- validating results
- calculating metrics
- checking reconstruction accuracy

It MUST NOT leak into the inference pipeline.

NETSCOPE-X must infer the topology from observations.

Never hardcode:

API-1 → Redis

into the inference engine simply because the test environment knows this relationship.

5. SAFETY BOUNDARY

All active traffic generation, failure injection, packet capture, routing experiments, and network manipulation must occur only inside a controlled laboratory environment that we own or explicitly configure for testing.

Do not target external systems.

Do not perform unauthorized scanning.

Do not perform destructive testing against production infrastructure.

The project should support safe simulation and controlled Docker/network-namespace experiments.

6. PREFERRED TECHNOLOGY STACK

Use technologies only when justified.

Preferred baseline:

Backend

- Python
- FastAPI

- Pydantic
- NetworkX
- NumPy
- Pandas
- SciPy

Network analysis

- Scapy
- tshark / Wireshark tooling where appropriate
- Linux networking tools
- optional eBPF only if justified and maintainable

Frontend

- React
- TypeScript
- Vite
- Tailwind CSS
- Cytoscape.js or another suitable graph visualization library

DO NOT use Streamlit for the primary interface.

Laboratory

- Docker
- Docker Compose
- Linux network namespaces where appropriate

Testing

- pytest
- Playwright
- frontend unit/integration testing
- API testing

Storage

Choose the minimum persistence architecture necessary.

Do not introduce PostgreSQL/Neo4j/Redis/etc. merely because they sound impressive.

Every infrastructure component must have a clear purpose.

7. ARCHITECTURAL PRINCIPLES

Use:

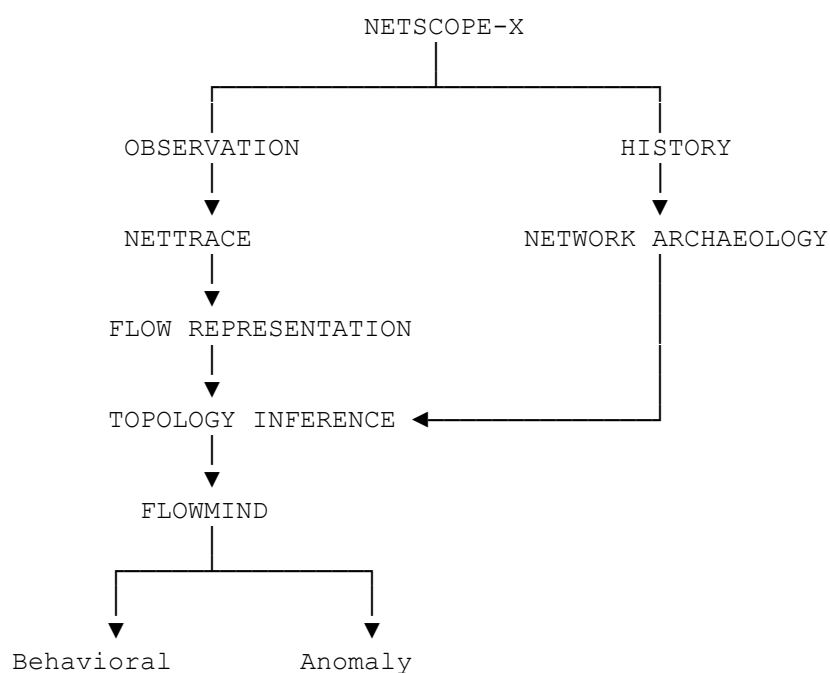
- modular architecture
- typed interfaces
- dependency injection where useful
- clear domain models
- deterministic processing wherever possible
- configuration-driven behavior
- structured logging
- versioned APIs
- reproducible experiments
- testable components

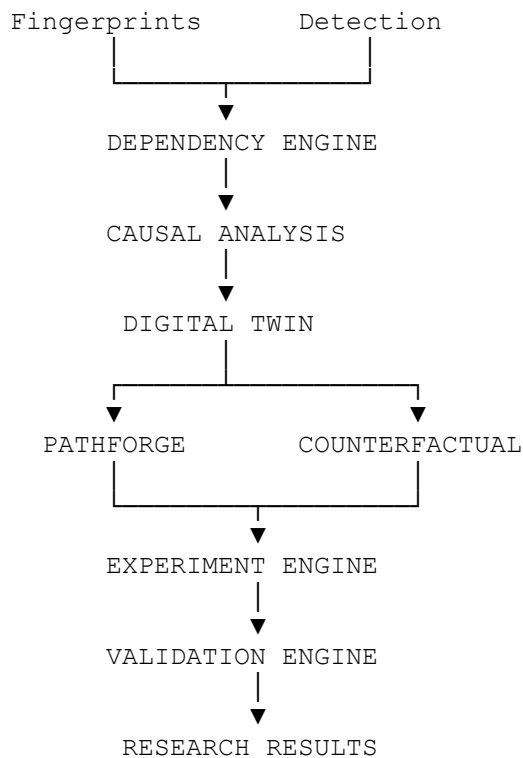
Avoid:

- giant files
- circular imports
- global state
- hidden side effects
- hardcoded paths
- hardcoded network topology
- unexplained magic numbers
- unnecessary abstractions
- overengineering without measurable benefit

8. CORE SYSTEM ARCHITECTURE

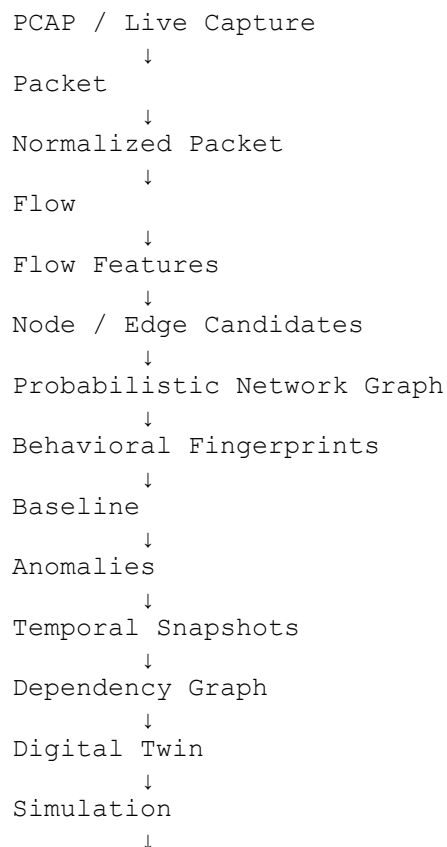
Target architecture:





9. DATA FLOW

The canonical data flow should be:



Counterfactual Scenario

↓

Experimental Validation

Each stage must have a defined input/output contract.

10. 69-PHASE EXECUTION PLAN

You MUST execute exactly these 69 phases.

Do NOT change the phase count.

Do NOT merge phases.

Do NOT silently skip phases.

Do NOT mark multiple phases complete with one vague statement.

PHASE 01 — RESEARCH PROBLEM FORMALIZATION

Define:

- exact problem
- assumptions
- observability model
- supported network environments
- limitations
- research boundaries
- intended users
- success criteria

Deliver:

`docs/research/problem_definition.md`

PHASE 02 — RESEARCH QUESTIONS AND HYPOTHESES

Define measurable research questions covering:

- topology reconstruction
- behavioral inference
- anomaly detection
- temporal change detection
- dependency inference
- failure propagation
- counterfactual simulation

Each question must have:

- hypothesis
- variables
- evaluation metric
- experiment design

PHASE 03 — SYSTEM REQUIREMENTS

Create:

- functional requirements
- non-functional requirements
- performance requirements
- reliability requirements
- security requirements
- reproducibility requirements

PHASE 04 — ARCHITECTURE AND DATA CONTRACTS

Define schemas for:

- packet
- flow
- node
- edge
- topology
- behavioral fingerprint
- anomaly
- snapshot
- dependency
- failure
- simulation
- experiment

- metric

PHASE 05 — ALGORITHM SELECTION

Evaluate candidate algorithms for:

- flow reconstruction
- role inference
- anomaly detection
- graph criticality
- path analysis
- dependency inference

Document:

- alternatives
- advantages
- limitations
- computational complexity
- selected method

PHASE 06 — REPRODUCIBLE DEVELOPMENT ENVIRONMENT

Create:

- Docker configuration
- Python environment
- frontend environment
- pinned dependencies
- development scripts

Fresh installation must work.

PHASE 07 — OBSERVABILITY FRAMEWORK

Implement:

- structured logs
 - request IDs
 - experiment IDs
 - module-level logging
 - error reporting
 - performance timing
-

PHASE 08 — CONFIGURATION AND SECRETS

Implement:

- `.env.example`
- environment configuration
- validation
- secret handling
- development/test/prod configuration

No secrets in source control.

PHASE 09 — API ARCHITECTURE

Design API contracts for:

```
/capture  
/flows  
/topology  
/behaviors  
/anomalies  
/history  
/dependencies  
/causal  
/simulation  
/counterfactual  
/experiments  
/metrics
```

PHASE 10 — RESEARCH ARTIFACT ARCHITECTURE

Define reproducible formats for:

- PCAP
 - flows
 - ground truth
 - graphs
 - snapshots
 - experiments
 - results
 - metrics
-

PHASE 11 — MULTI-TIER NETWORK LABORATORY

Build the controlled Docker network.

Minimum:

Client
Gateway
Load Balancer
API-1
API-2
Redis
Database
Worker
DNS
External-service simulator

PHASE 12 — NETWORK NAMESPACE ISOLATION

Implement controlled network boundaries and verify routing.

PHASE 13 — ROUTING LABORATORY

Create multiple routes and controlled routing changes.

Verify actual packet paths.

PHASE 14 — TRAFFIC WORKLOAD GENERATOR

Implement reproducible:

- normal
- burst
- periodic
- concurrent
- idle
- degraded

traffic.

PHASE 15 — PROTOCOL WORKLOAD GENERATOR

Generate realistic:

- TCP
- UDP
- DNS
- HTTP
- TLS metadata
- database
- cache

traffic.

PHASE 16 — GROUND-TRUTH GENERATOR

Automatically generate authoritative:

- nodes
- edges
- roles
- services
- routes
- expected paths

PHASE 17 — GROUND-TRUTH INTEGRITY

Version and hash ground-truth artifacts.

Prevent accidental contamination of inference.

PHASE 18 — SCENARIO GENERATOR

Generate multiple controlled network architectures.

Examples:

- simple chain
 - star
 - multi-tier
 - redundant
 - multi-path
 - dynamic service network
-

PHASE 19 — TRAFFIC REPLAY ENGINE

Support deterministic replay of recorded workloads.

PHASE 20 — OBSERVATORY VALIDATION

Before NETTRACE receives data, verify that the laboratory itself behaves correctly.

Validate:

- expected connectivity
- expected routes
- expected services

- expected traffic
-

PHASE 21 — HIGH-FIDELITY PACKET CAPTURE

Implement:

- PCAP ingestion
- controlled live capture

Do not capture outside authorized environments.

PHASE 22 — PACKET NORMALIZATION

Normalize:

- timestamps
 - IPs
 - ports
 - protocol
 - packet size
 - direction
 - transport information
-

PHASE 23 — FIVE-TUPLE FLOW RECONSTRUCTION

Implement bidirectional flow tracking.

Handle:

- TCP
 - UDP
-

PHASE 24 — TCP STATE RECONSTRUCTION

Handle:

- SYN
 - SYN/ACK
 - ACK
 - FIN
 - RST
 - retransmissions
 - partial sessions
-

PHASE 25 — UDP SESSION MODELING

Create temporal UDP sessions based on:

- endpoints
 - ports
 - timing windows
 - packet sequences
-

PHASE 26 — PROTOCOL FINGERPRINTING

Implement protocol inference using observable evidence.

Do not claim unsupported protocol coverage.

PHASE 27 — ENCRYPTED TRAFFIC METADATA ANALYSIS

Analyze permitted metadata such as:

- TLS versions
- connection duration
- packet sizes

- timing
- endpoint relationships

Do not decrypt traffic.

PHASE 28 — ADVANCED FLOW FEATURES

Generate:

- packet statistics
 - byte statistics
 - duration
 - burstiness
 - inter-arrival times
 - directionality
 - destination diversity
 - port diversity
 - connection persistence
-

PHASE 29 — NODE DISCOVERY

Infer active nodes exclusively from observations.

PHASE 30 — EDGE DISCOVERY

Infer communication relationships.

PHASE 31 — PROBABILISTIC EDGE CONFIDENCE

Represent:

Node A → Node B

with:

- confidence
- evidence
- observation count
- timestamps
- protocols

Do not use arbitrary confidence values.

PHASE 32 — PROBABILISTIC TOPOLOGY RECONSTRUCTION

Produce the complete inferred network graph.

Output:

nodes
edges
confidence
evidence
timestamps
protocols

Compare against ground truth.

PHASE 33 — BEHAVIORAL FEATURE STORE

Create reusable behavioral feature representations.

PHASE 34 — MULTI-WINDOW BEHAVIOR MODELING

Model behavior over:

- short
- medium
- long

observation windows.

PHASE 35 — NODE BEHAVIORAL FINGERPRINTS

Generate fingerprints from:

- traffic
- ports
- protocols
- timing
- destinations
- directionality
- persistence

PHASE 36 — SERVICE ROLE INFERENCE

Infer roles such as:

Client
Gateway
API
Database
Cache
DNS
Worker
Load Balancer
Unknown

PHASE 37 — UNCERTAINTY-AWARE CLASSIFICATION

Represent uncertainty.

Example:

Database: 72%
Cache: 21%
Unknown: 7%

Confidence must be calibrated.

PHASE 38 — BEHAVIORAL BASELINE

Build a normal-behavior model from historical observations.

PHASE 39 — CONCEPT DRIFT DETECTION

Distinguish:

temporary anomaly

from:

persistent behavioral evolution

PHASE 40 — MULTI-DIMENSIONAL ANOMALY DETECTION

Detect deviations in:

- traffic volume
 - destinations
 - ports
 - protocols
 - timing
 - topology
 - behavior
-

PHASE 41 — EXPLAINABLE ANOMALIES

Every anomaly must provide actual evidence.

Example:

Node: API-2

New destination:
X

Historical destinations:
4

Current destinations:
9

New port:
4444

Evidence:
...

PHASE 42 — FLOWMIND EVALUATION

Measure:

- precision
 - recall
 - F1
 - false-positive rate
 - false-negative rate
 - detection latency
-

PHASE 43 — TEMPORAL GRAPH MODEL

Represent network as:

$G(t)$

instead of only a static graph.

PHASE 44 — NETWORK SNAPSHOT ENGINE

Generate versioned network snapshots.

PHASE 45 — GRAPH DIFFERENCE ENGINE

Detect:

- node additions
 - node removals
 - edge additions
 - edge removals
 - attribute changes
-

PHASE 46 — BEHAVIORAL EVOLUTION TRACKING

Track behavioral changes for individual nodes.

PHASE 47 — TOPOLOGY EVENT TIMELINE

Create a chronological network event stream.

PHASE 48 — CHANGE ATTRIBUTION

Associate changes with:

- observation evidence
- timestamps
- affected flows
- affected nodes

Do not claim a causal explanation without sufficient evidence.

PHASE 49 — HISTORICAL INVESTIGATION ENGINE

Support queries such as:

What changed between 10:30 and 11:15?

Return structured results.

PHASE 50 — COMMUNICATION VS DEPENDENCY

Clearly distinguish:

A communicates with B

from:

A depends on B

Communication alone must not automatically imply dependency.

PHASE 51 — DEPENDENCY STRENGTH

Estimate dependency strength using:

- frequency
 - persistence
 - directionality
 - temporal relationships
 - traffic characteristics
-

PHASE 52 — TEMPORAL PRECEDENCE ANALYSIS

Study whether changes in one component consistently precede changes in another.

PHASE 53 — CAUSAL CANDIDATE GENERATION

Generate candidate causal relationships.

Do not equate correlation with causation.

PHASE 54 — FAILURE PROPAGATION GRAPH

Represent:

```
Primary failure
  ↓
Secondary impact
  ↓
Tertiary impact
```

PHASE 55 — CRITICALITY ANALYSIS

Analyze:

- degree
- betweenness
- articulation points
- path dependency
- connectivity

Document why each metric is relevant.

PHASE 56 — CAUSAL EVIDENCE REPORT

For every inferred dependency or propagation relationship provide:

```
relationship
evidence
```

confidence
counter-evidence
limitations

PHASE 57 — DIGITAL TWIN

Build a computational model combining:

Topology
Behavior
History
Dependencies
Routing
State

PHASE 58 — DIGITAL TWIN SYNCHRONIZATION

Update the twin from new observations.

Handle:

- additions
 - removals
 - behavior changes
 - confidence changes
-

PHASE 59 — FAILURE INJECTION FRAMEWORK

Support controlled simulation of:

node failure
edge failure
latency
packet loss
bandwidth reduction
service degradation

PHASE 60 — DYNAMIC PATH ENGINE

Implement:

- shortest paths
 - alternate paths
 - path costs
 - route changes
 - disconnected components
-

PHASE 61 — FAILURE PROPAGATION SIMULATOR

Simulate:

```
failure
↓
dependency propagation
↓
routing impact
↓
service impact
```

PHASE 62 — RESILIENCE QUANTIFICATION

Calculate measurable indicators:

- connectivity
 - reachable-node ratio
 - affected services
 - path degradation
 - bottleneck emergence
 - alternative-path availability
-

PHASE 63 — DIGITAL TWIN VALIDATION

Compare simulation predictions against actual controlled experiments.

Do not claim correctness without measurement.

PHASE 64 — COUNTERFACTUAL SCENARIO LANGUAGE

Create structured scenarios:

```
REMOVE node  
REMOVE edge  
INCREASE latency  
REDUCE bandwidth  
INCREASE traffic  
ADD route
```

PHASE 65 — COUNTERFACTUAL GRAPH ENGINE

Generate isolated alternate network states.

Never mutate the original baseline.

PHASE 66 — COUNTERFACTUAL IMPACT ANALYSIS

Compare:

```
baseline  
vs  
counterfactual
```

for:

- paths
 - connectivity
 - latency
 - affected services
 - bottlenecks
 - propagation
-

PHASE 67 — AUTOMATIC EXPERIMENT GENERATOR

Use structural information to propose controlled experiments.

Example:

High-criticality node detected.

Suggested experiment:
Remove node.

Reason:
High betweenness
+
high dependency
+
few alternate paths

Every recommendation must be based on measurable system evidence.

PHASE 68 — COMPLETE RESEARCH VALIDATION

This is the main research evaluation phase.

Build an experimental matrix covering:

Topology complexity

small
medium
large
multi-path
multi-service
dynamic

Observation completeness

100%
90%
75%
50%
25%

Topology reconstruction

Measure:

node precision
node recall
edge precision
edge recall
F1
graph similarity

Role inference

Measure:

accuracy
precision
recall
F1
calibration

Anomaly detection

Measure:

precision
recall
F1
false positive rate
detection latency

Temporal analysis

Measure:

change precision
change recall
detection latency

Causal analysis

Measure:

dependency precision
dependency recall
impact prediction
propagation accuracy

PathForge

Measure:

path prediction
affected-node prediction

connectivity prediction

Counterfactuals

Compare:

predicted outcome
vs
actual controlled experiment

Ablation studies

At minimum test:

Full system
vs
without temporal features

Full system
vs
without behavioral features

Full system
vs
without dependency weighting

Full system
vs
without confidence modeling

Every result must be generated from actual experiments.

PHASE 69 — FINAL HARDENING AND RELEASE

Perform complete:

Backend testing

- unit
- integration
- API
- concurrency
- failure recovery
- malformed input
- large PCAP
- incomplete PCAP

Network testing

Test:

- missing packets
- duplicated packets
- out-of-order packets
- partial flows
- short flows
- long flows
- high-volume traffic
- unknown protocols
- encrypted traffic metadata
- NAT scenarios
- dynamic endpoints
- IP reuse

Frontend testing

Use a REAL Chrome browser.

Test:

- navigation
- topology rendering
- large graphs
- filters
- node inspection
- edge inspection
- timeline
- anomalies
- simulation
- counterfactuals
- errors
- empty states
- loading states
- responsive behavior

Final cleanup

Remove:

- dead code
- debug prints
- temporary files
- unused imports
- fake data
- accidental secrets

- TODOs that should have been resolved

Documentation

Generate/update:

```
README.md
ARCHITECTURE.md
API.md
DEVELOPMENT.md
TESTING.md
EXPERIMENTS.md
LIMITATIONS.md
DEPLOYMENT.md
```

Research documentation:

```
RESEARCH_QUESTIONS.md
METHODOLOGY.md
DATASET.md
GROUND_TRUTH.md
EVALUATION.md
ABLATION_STUDY.md
THREATS_TO_VALIDITY.md
```

11. FRONTEND REQUIREMENTS

The frontend must NOT look like a generic admin dashboard.

Use a visual identity appropriate to:

- network observability
- distributed systems
- network intelligence
- digital twins

Preferred visual concept:

```
dark control-center interface
+
high-contrast network graph
+
subtle grid
+
clear typography
+
technical visualization
+
minimal neon accents
```

Avoid:

- excessive gradients
- excessive glassmorphism
- random animations
- generic SaaS cards
- excessive rounded cards
- boring full-white layouts

The network graph must be the visual centerpiece.

12. REQUIRED FRONTEND AREAS

Implement at least:

Overview

Show:

- discovered nodes
- active connections
- anomalies
- topology confidence
- network health
- recent changes

Topology Explorer

Interactive graph.

Node Investigation

Show:

- identity
- inferred role
- confidence
- connections
- protocols
- behavioral fingerprint
- anomalies
- criticality

Timeline

Show topology evolution.

Anomaly Investigation

Show actual evidence.

Causal Analysis

Show:

```
source
↓
relationship
↓
evidence
↓
potential impact
```

Simulation

Allow controlled scenarios.

Counterfactual

Allow:

"What if...?"

Experiment Lab

Show:

- experiment
- hypothesis
- setup
- result
- metrics
- comparison

13. API DESIGN PRINCIPLES

All APIs must:

- validate input
- return consistent errors
- use typed schemas
- avoid leaking internal exceptions
- support pagination where needed

- provide meaningful status codes
 - document request/response schemas
-

14. PERFORMANCE REQUIREMENTS

Do not optimize prematurely.

First establish correctness.

Then measure:

- packet processing throughput
- flow processing time
- graph construction time
- memory usage
- API latency
- frontend graph rendering performance
- simulation time

Identify bottlenecks using measurements.

Do not claim scalability without benchmarks.

15. ERROR HANDLING

Handle:

- corrupted PCAP
- malformed packets
- missing fields
- incomplete flows
- unknown protocols
- duplicate packets
- out-of-order packets
- invalid scenario definitions
- disconnected graphs
- empty datasets
- huge datasets
- unavailable network interfaces

The application must fail gracefully.

16. SECURITY REQUIREMENTS

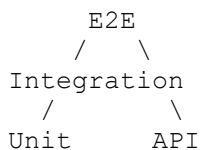
Implement appropriate:

- input validation
- command sanitization
- upload validation
- path validation
- resource limits
- safe subprocess execution
- controlled packet capture
- authentication if required
- API abuse protection where appropriate

Never execute arbitrary user-supplied shell commands.

17. TESTING PYRAMID

Implement:



Tests should cover:

Unit

Individual algorithms.

Integration

Module-to-module behavior.

End-to-end

Actual:

```
PCAP
↓
NETTRACE
↓
FLOWMIND
↓
Archaeology
↓
```

Causal
↓
Simulation

Browser

Use real Chrome with Playwright or available browser automation.

18. CHROME TESTING REQUIREMENT

For the final frontend validation:

DO NOT rely only on:

- DOM snapshots
- static HTML
- component tests
- screenshots
- mocked browser responses

Use a real Chrome browser.

Verify actual user workflows.

Minimum workflow:

```
Open application
↓
Load network capture
↓
Wait for processing
↓
Open topology
↓
Select node
↓
Inspect behavior
↓
Open anomaly
↓
Open timeline
↓
Run simulation
↓
Create counterfactual
↓
Inspect results
```

Take screenshots only as evidence after the workflow actually works.

19. TEST DATA REQUIREMENT

Create multiple datasets.

At minimum:

```
dataset_small  
dataset_medium  
dataset_large  
dataset_dynamic  
dataset_failure  
dataset_anomaly  
dataset_incomplete  
dataset_noisy
```

Every dataset should have:

- description
 - generation procedure
 - ground truth
 - expected properties
 - version
 - checksum
-

20. RESEARCH REPRODUCIBILITY

Every experiment must have:

```
experiment_id  
dataset_version  
code_version  
configuration  
random_seed  
timestamp  
environment  
parameters  
results
```

Experiments must be rerunnable.

21. NO FAKE METRICS

Never produce:

Accuracy: 97.4%

unless it came from actual execution.

If performance is poor, report poor performance.

If the algorithm fails under 25% observability, report that.

A scientifically honest limitation is better than fabricated success.

22. ALGORITHM TRANSPARENCY

For every important algorithm document:

- input
 - output
 - assumptions
 - mathematical idea
 - complexity
 - failure cases
 - limitations
 - why selected
-

23. AI/LLM USAGE

If an LLM is added:

It may be used for:

- natural-language explanations
- querying structured network state
- generating investigation summaries
- explaining simulation results

It must NOT:

- fabricate topology
- invent packet evidence
- invent metrics
- replace core topology inference
- replace graph algorithms
- silently make unsupported causal claims

Example:

User:

Why is API-2 critical?

The system should retrieve:

```
betweenness
dependency strength
paths
observed traffic
failure simulations
```

and then generate an explanation from those actual values.

24. REQUIRED FINAL DEMONSTRATION

The final demo should tell a coherent story.

Start with:

UNKNOWN NETWORK

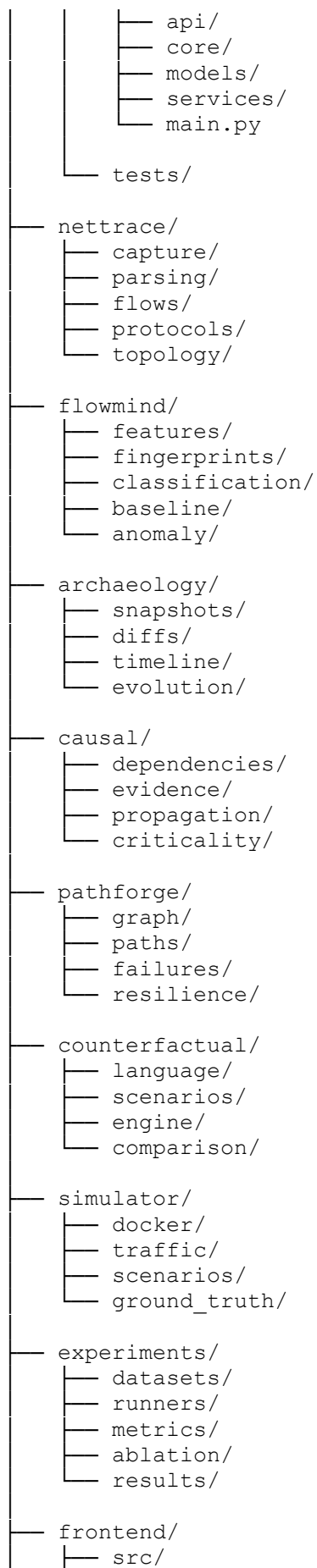
Then:

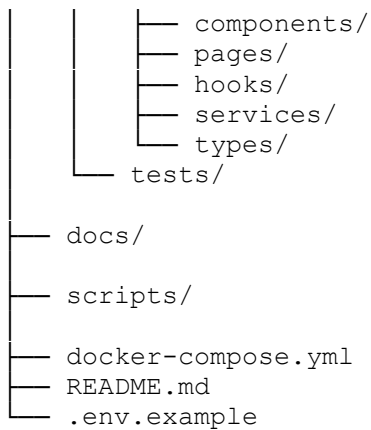
1. Capture traffic
 2. Discover nodes
 3. Reconstruct topology
 4. Infer service roles
 5. Establish behavioral baseline
 6. Detect behavioral change
 7. Show topology evolution
 8. Infer dependencies
 9. Identify critical components
 10. Create digital twin
 11. Inject controlled failure
 12. Simulate propagation
 13. Compare alternative paths
 14. Ask a counterfactual
 15. Run the real experiment
 16. Compare prediction vs actual
 17. Display research metrics
-

25. REQUIRED PROJECT STRUCTURE

Use a maintainable structure similar to:

```
netscope-x/
|
├─ backend/
|   └─ app/
```





Adapt this structure if inspection shows a better architecture.

Do not blindly create unnecessary directories.

26. PHASE COMPLETION PROTOCOL

After EVERY phase, stop and produce a report.

Use exactly this structure:

PHASE XX — <NAME>

STATUS:

COMPLETE / BLOCKED

OBJECTIVE:

...

IMPLEMENTED:

...

FILES CREATED:

...

FILES MODIFIED:

...

COMMANDS EXECUTED:

...

TESTS EXECUTED:

...

ACTUAL RESULTS:

...

FAILURES FOUND:

...

FIXES APPLIED:

...

VERIFICATION:

...

KNOWN LIMITATIONS:

...

NEXT PHASE:

...

Do not write:

"Everything works."

Provide actual evidence.

27. FAILURE RECOVERY PROTOCOL

If something fails:

```
Failure
↓
Read actual error
↓
Identify root cause
↓
Inspect relevant source
↓
Implement correction
↓
Run targeted test
↓
Run regression tests
↓
Verify
```

Do NOT:

```
Failure
↓
Ignore
↓
Continue
```

28. CONTEXT MANAGEMENT

This is a 69-phase project.

You may encounter context limitations.

If your context becomes large:

1. Preserve the current architecture.
2. Update project documentation.
3. Create a project state file.
4. Record completed phases.
5. Record unresolved issues.
6. Record architectural decisions.
7. Continue from the exact next incomplete phase.

Create:

```
docs/PROJECT_STATE.md
```

Keep it continuously updated.

29. PROJECT STATE FILE

Maintain:

```
Current phase:
Completed phases:
Blocked phases:
Known bugs:
Architecture decisions:
Current test status:
Current datasets:
Current metrics:
Pending work:
```

Never lose track of the phase number.

30. GIT DISCIPLINE

Use meaningful commits.

Suggested:

```
phase-01-research-definition
phase-02-research-questions
phase-03-system-requirements
...
```

Do not create meaningless commits such as:

```
update
fix
```

changes
stuff

Do not rewrite history unnecessarily.

31. DOCUMENTATION DISCIPLINE

Documentation must evolve alongside implementation.

Do not wait until Phase 69 to document everything.

Every significant architectural decision should be documented when made.

32. NO PREMATURE COMPLETION

A phase is NOT complete merely because:

- code exists
- imports succeed
- the server starts
- a screenshot looks correct
- a mock returns expected data

A phase is complete only when its actual acceptance criteria are verified.

33. ACCEPTANCE TEST FOR CORE SYSTEM

Eventually the system must successfully perform:

```
Controlled network
  ↓
Generate real traffic
  ↓
Capture traffic
  ↓
Parse packets
  ↓
Reconstruct flows
  ↓
Infer nodes
  ↓
Infer edges
```

↓
Build probabilistic topology
↓
Infer service roles
↓
Build behavioral baseline
↓
Detect controlled anomaly
↓
Detect topology change
↓
Infer dependencies
↓
Construct digital twin
↓
Simulate failure
↓
Predict impact
↓
Execute controlled failure
↓
Compare prediction with reality
↓
Generate metrics

This end-to-end test is mandatory.

34. QUALITY BAR

The final system should demonstrate:

Correctness

Results correspond to actual observations.

Reproducibility

Experiments can be repeated.

Explainability

Important conclusions have evidence.

Robustness

Malformed and incomplete inputs do not crash the system.

Scientific validity

Claims are supported by experiments.

Engineering quality

Code is modular and maintainable.

UX quality

The frontend is usable in a real browser.

35. WHAT NOT TO DO

Do NOT turn this into:

- generic network monitoring
- simple packet sniffer
- generic intrusion detection
- generic ML anomaly detector
- generic chatbot
- generic admin dashboard
- fake digital twin
- hardcoded topology visualizer
- static simulation demo

The project must preserve its central research identity:

Infer → Model → Reason → Simulate → Validate

36. FINAL SUCCESS CRITERIA

The project is successful only if it can demonstrate all of the following:

A. Observation

Actual traffic can be processed.

B. Reconstruction

Topology can be inferred without ground-truth leakage.

C. Behavioral intelligence

Node behavior can be modeled.

D. Temporal intelligence

Topology and behavior changes can be detected.

E. Dependency reasoning

Observed communication can be analyzed for probable dependencies.

F. Digital twin

The inferred system can be represented computationally.

G. Simulation

Controlled failures can be simulated.

H. Counterfactuals

Hypothetical scenarios can be evaluated.

I. Validation

Predictions can be compared with real controlled experiments.

J. Research evidence

Results are reproducible and quantitatively evaluated.

37. STARTING INSTRUCTION

Do NOT immediately begin writing large amounts of code.

Start by:

1. Inspecting the repository.
2. Inspecting the existing environment.
3. Checking available runtimes/tools.
4. Checking whether Docker is installed.
5. Checking whether Chrome is available for testing.
6. Checking Python and Node versions.
7. Inspecting existing files.
8. Creating the initial project state.
9. Completing Phase 01 only.

After Phase 01:

- execute its acceptance criteria
- produce the Phase 01 report

- `update docs/PROJECT_STATE.md`

Then proceed to Phase 02.

38. ABSOLUTE EXECUTION RULE

DO NOT JUMP AHEAD.

Execute:

```
Phase 01
↓
verify
↓
Phase 02
↓
verify
↓
Phase 03
↓
verify
...
↓
Phase 69
```

If a later phase requires a design decision that should have been made earlier, update the relevant documentation and explain the change.

Never silently alter the architecture.

39. FINAL INSTRUCTION TO CLAUDE

You are responsible for the engineering outcome, not merely the code output.

Think like:

```
Senior Software Architect
+
Network Systems Engineer
+
Distributed Systems Engineer
+
Research Engineer
+
QA Engineer
+
DevOps Engineer
```

Do not optimize for the amount of code produced.

Optimize for:

CORRECTNESS
+
REPRODUCIBILITY
+
MEASURABILITY
+
EXPLAINABILITY
+
ENGINEERING QUALITY

The final project must be something that can be demonstrated to technically strong interviewers and defended academically.

When you encounter a choice between:

quick implementation

and

correct, testable, explainable implementation

choose the second.

When you encounter a choice between:

impressive-looking fabricated result

and

honest measured result

choose the measured result.

When uncertain:

INSPECT
→
MEASURE
→
IMPLEMENT
→
TEST
→
VERIFY

Never guess when the repository or runtime can provide evidence.

Begin now with **Phase 01 — Research Problem Formalization** and do not proceed to Phase 02 until Phase 01 has been genuinely completed and verified.